Day 4 - Convolutional Neural Networks

Small filters that slide over images - and the start of the challenge

Annika Stiehl · Deep Learning · M.Sc. · SRH

Today

Time	
09:00 - 10:30	Convolutions: a small filter slides over the image • padding, stride, pooling • a whole CNN
10:45 - 12:15	Deeper is not always better • object detection in a nutshell • sanity checks before training
13:00 - 14:30	Lab 4: build your own CNN (task sheet at the end of the PDF)
14:45 - 16:00	**The challenge**: task, rules, the starter code together, teams • Q&A

💻 Today we code during the lecture: open `day4a_convolutions` and `day4b_architectures` from the course repository in Colab (CPU is enough).

Last time

- the **training loop** has knobs: loss, optimiser, learning rate, schedule, batch size, epochs
- **overfitting**: training error low, validation error high → weight decay, dropout, early stopping, augmentation
- **evaluation**: split by recording / patient, never one metric alone, several seeds

Today: so far every model got a flat list of numbers. But images have **structure**: neighbouring pixels belong together. Convolutions use this structure.

Convolutions

A small filter slides over the image

Why not an MLP for images?

- a small colour image: 32 × 32 × 3 = 3 072 numbers → one hidden layer with 1 000 neurons = **3 million weights**
- a photo of 224 × 224 pixels: 150 528 numbers → **150 million weights** in the first layer alone
- move the object by 3 pixels → a completely different list of numbers: the MLP must learn every object at every position again
- the MLP does not know which pixels are neighbours

❓ Think about it

We shuffle the pixels of every image in the same fixed way and train an MLP from scratch. Does its accuracy change? Could you still recognise the images?

Try it: filters on an image

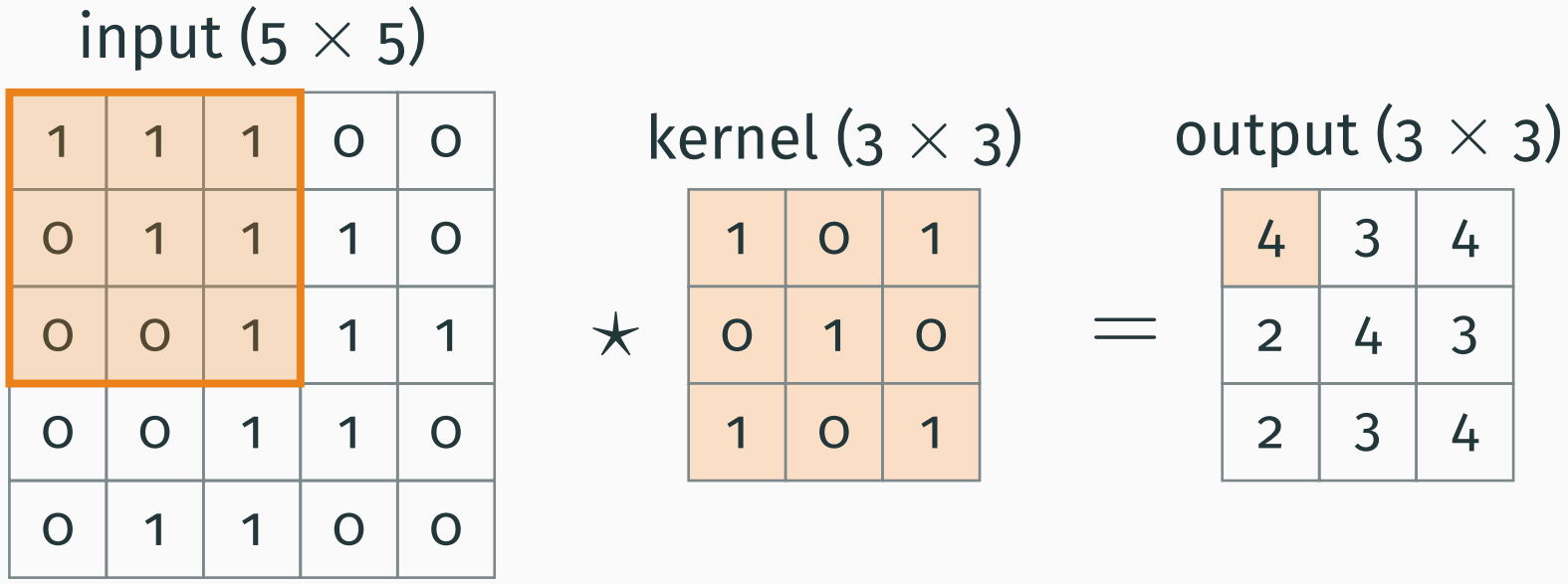
setosa.io/ev/image-kernels - an image, a 3 × 3 grid of numbers (the **kernel**), and the result

1. choose **blur**: what do the numbers in the grid look like?
2. choose **sharpen**, then **outline**: what is highlighted?
3. scroll to the image with the moving 3 × 3 window: how is one output pixel computed?

👥 Your turn (8 min)

In pairs, in your browser. Keep the answer to question 3 - it is the next slide.

A convolution: a small filter slides over the image

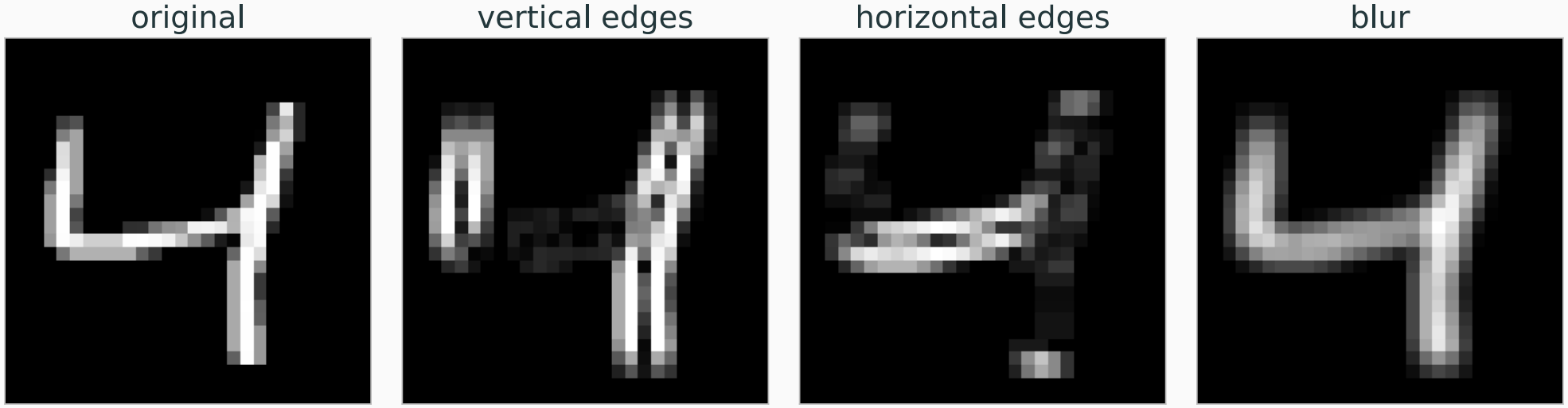


- put the kernel on the image, **multiply** the numbers that lie on top of each other, **add** them up → one output value
- slide one step to the right and do it again - row by row
- top left: 1·1 + 1·0 + 1·1 + 0·0 + 1·1 + 1·0 + 0·1 + 0·0 + 1·1 = **4**

❓ Think about it

Compute the output value in the middle (row 2, column 2) yourself. Which value do you get?

Hand-made filters on a real image



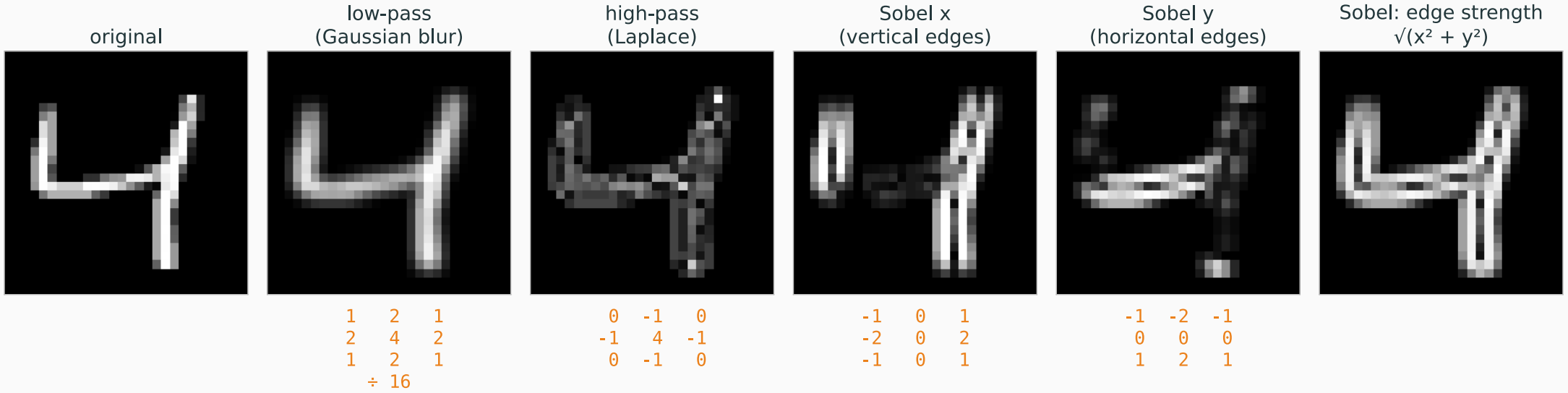

```
-1   0   1
-1   0   1
-1   0   1
```

```
-1   -1   -1
 0    0    0
 1    1    1
```

```
1/9   1/9   1/9
1/9   1/9   1/9
1/9   1/9   1/9
```

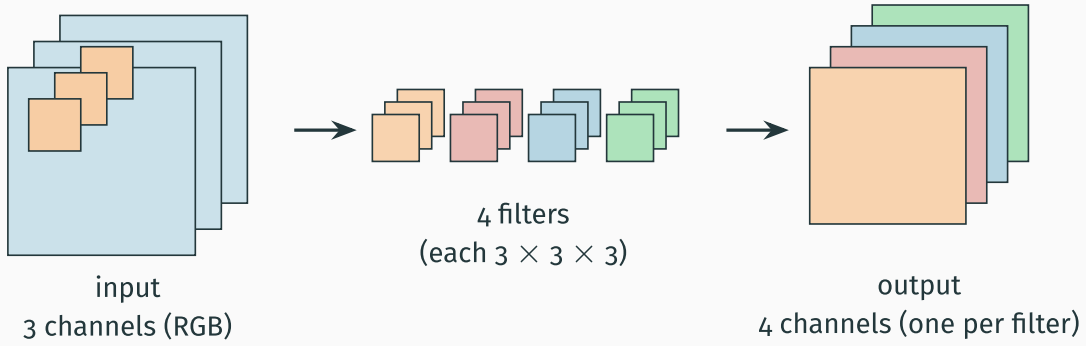
- a kernel `[-1 0 1]` (three rows, the **Prewitt** filter) reacts where the image changes from **left to right** → vertical edges
- turned by 90° → horizontal edges; all values 1/9 → blur (the average of the neighbourhood)
- a CNN **learns** its kernels itself - the first layer usually ends up with edge and colour detectors like these

Classic filters have names: low-pass, high-pass, Sobel



filter	what it does	what it keeps
low-pass (e.g. Gaussian blur)	averages each pixel with its neighbours	slow changes: large areas - removes noise and fine detail
high-pass (e.g. Laplace)	pixel minus its neighbours	fast changes: edges and fine detail - flat areas become 0
Sobel x / y	difference in one direction, slightly smoothed	edges in one direction; x and y together give the edge strength

Colour images and many filters: channels

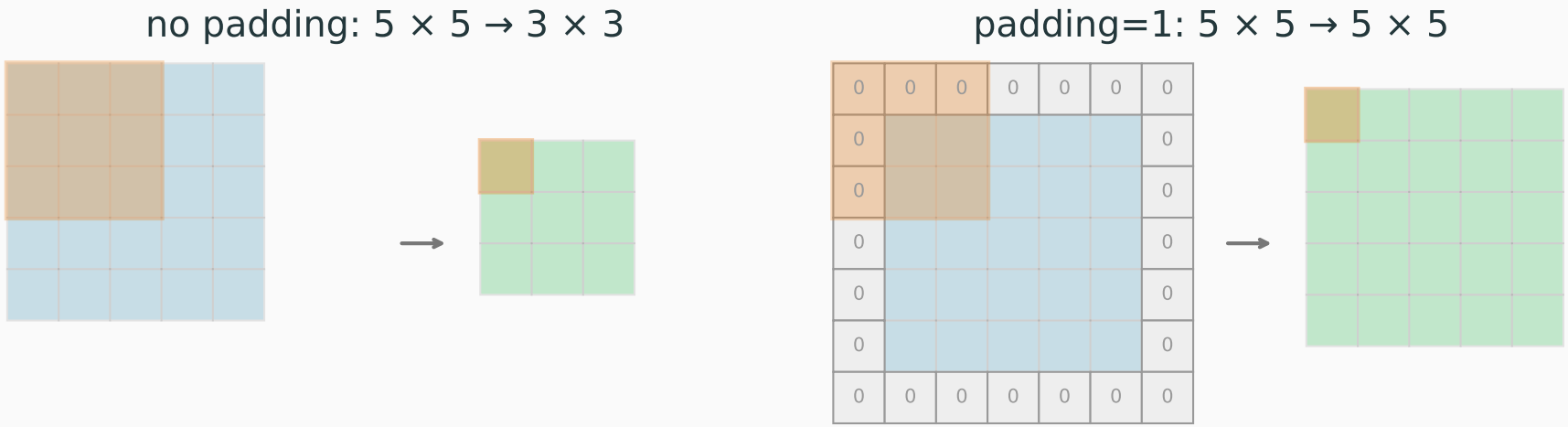


1. **input**: a colour image = **3 channels** - one grid each for red, green, blue
2. **one filter** = a small block of 3 × 3 × 3 weights: it looks at the same 3 × 3 spot in all three colours and adds everything up → **one** number
3. the filter slides over the image → **one output channel** (a "feature map")
4. **4 filters → 4 output channels** - one per filter

```
nn.Conv2d(3, 4, kernel_size=3)  # 3 in, 4 out
# weights: 4 × (3·3·3 + 1) = 112
```

Real CNNs use more filters (16, 32, 64, ...) - the idea is the same.

Padding: keep the size

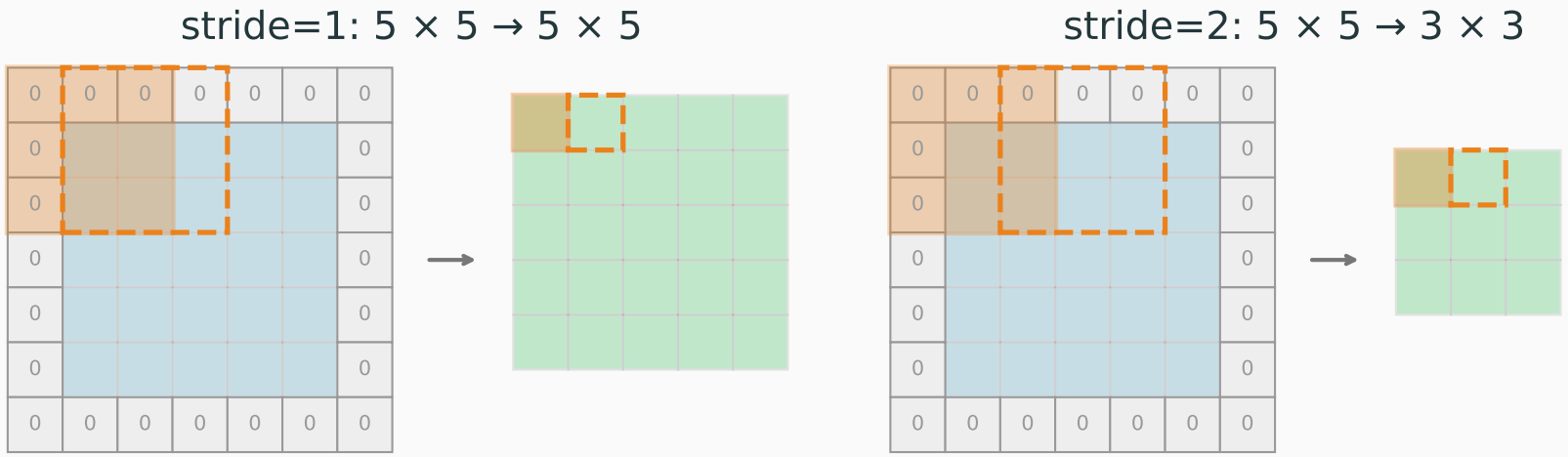


grey 0 = padding (a border of zeros); orange = the 3 × 3 window and the output value it produces

- **without padding** every 3 × 3 layer makes the image 2 pixels smaller (5 → 3) - the window cannot sit on the border
- **padding** = a border of zeros around the image: now the window can also sit on the border pixels
- `padding=1` keeps the size for a 3 × 3 kernel - in general `padding = kernel // 2` (5 × 5 kernel: `padding=2`)

```
nn.Conv2d(1, 16, kernel_size=3, padding=0)   # 28 × 28 → 26 × 26
nn.Conv2d(1, 16, kernel_size=3, padding=1)   # 28 × 28 → 28 × 28
```

Stride: how far the window jumps



orange = first window and its output value; dashed = the next window position and its output value

- **stride** = how many pixels the window jumps to the next position
- `stride=1`: every position → the size stays (with padding) · `stride=2`: every second position → about **half** the size
- like max pooling, a way to make the image smaller - but here the network learns how
- rule: output = (input + 2 · padding − kernel) / stride + 1, rounded down → (28 + 2 − 3) / 2 + 1 = 14.5 → **14**

```
nn.Conv2d(1, 16, kernel_size=3, padding=1, stride=1)   # 28 × 28 → 28 × 28
nn.Conv2d(1, 16, kernel_size=3, padding=1, stride=2)   # 28 × 28 → 14 × 14
```

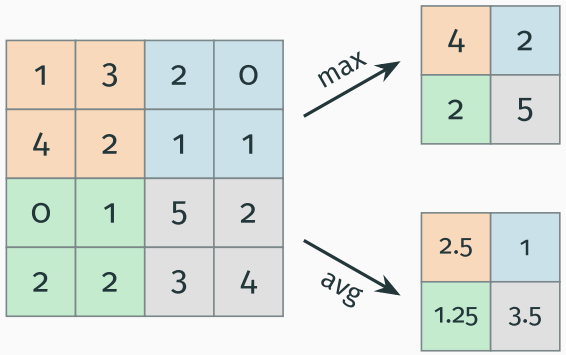
In PyTorch: shapes first

```
x = torch.randn(8, 1, 28, 28)              # (batch, channels, height, width) - channels first!
conv = nn.Conv2d(1, 16, kernel_size=3, padding=1)
print(conv(x).shape)                       # torch.Size([8, 16, 28, 28])

pool = nn.MaxPool2d(2)
print(pool(conv(x)).shape)                 # torch.Size([8, 16, 14, 14])
```

- PyTorch order: **(batch, channels, height, width)** - a normal image file is height × width × channels
- most bugs in CNN code are wrong shapes → **print the shape after every layer**

Pooling: make the image smaller

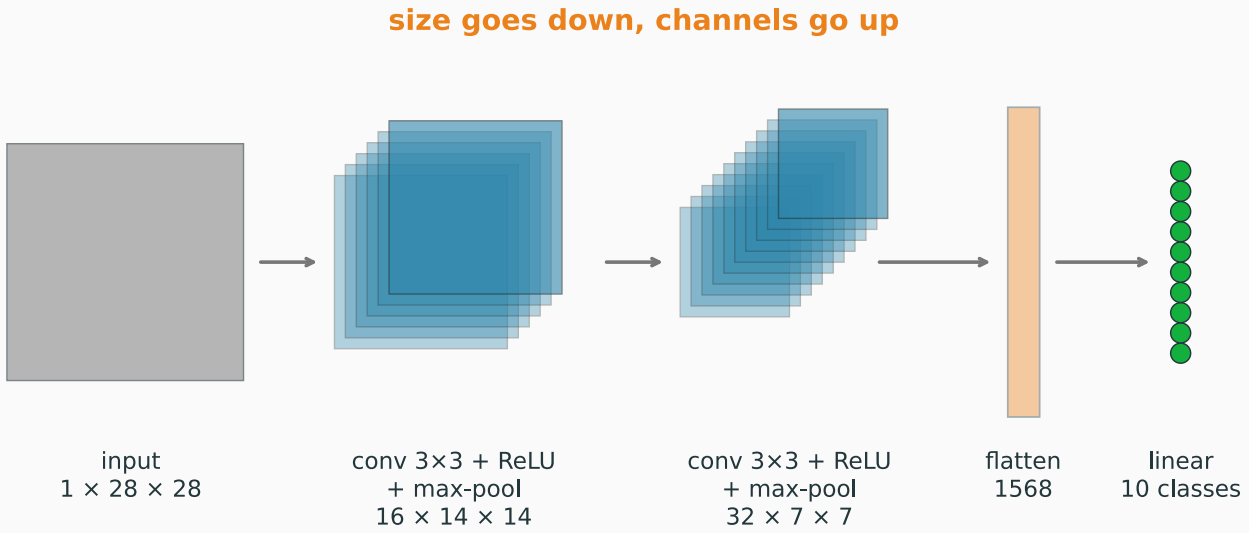


2 × 2 windows, stride 2

- **max pooling**: keep the largest value of every 2 × 2 window - "is the feature somewhere here?"
- **average pooling**: keep the mean
- halves height and width, keeps the channels - no weights to learn
- a small shift often does not change the maximum → more robust

```
x = torch.randn(8, 16, 28, 28)        # 8 images, 16 feature maps of 28 × 28
nn.MaxPool2d(2)(x)                    # → (8, 16, 14, 14)  max of every 2 × 2 window
nn.AvgPool2d(2)(x)                    # → (8, 16, 14, 14)  mean of every 2 × 2 window
nn.AdaptiveAvgPool2d(1)(x)            # → (8, 16, 1, 1)    global average pooling
nn.AdaptiveMaxPool2d(1)(x)            # → (8, 16, 1, 1)    global max pooling
nn.AdaptiveAvgPool2d((7, 7))(x)       # → (8, 16, 7, 7)    any output size you want
```

Putting it together: a small CNN




```
model = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),   # 16 filters → 16 feature maps of 14 × 14
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 32 filters → 32 feature maps of 7 × 7
    nn.Flatten(),                                                 # 32 × 7 × 7 = 1 568 numbers
    nn.Linear(32 * 7 * 7, 10))                                    # 10 classes
```

20 490 weights - the MLP from yesterday had **235 146** (and is worse, as you will see in the lab).

Try it: CNN Explainer

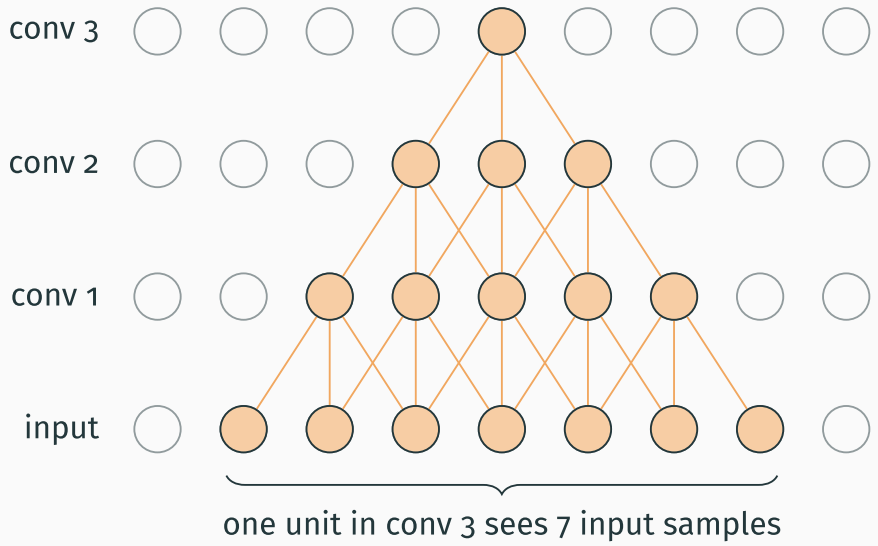
poloclub.github.io/cnn-explainer - a small trained CNN, live in your browser

1. pick an image at the top. Follow it from the left (the colours) to the right (the classes)
2. **click** on one square in the first conv layer: which kernel is used, and how is the value computed?
3. what happens to the size of the images from left to right? Where is the max pooling?

👥 Your turn (10 min)

In pairs, in your browser. Find one thing that surprised you.

Each layer sees a bit more: the receptive field



- one value in layer 1 sees 3 input pixels, in layer 2 already 5, in layer 3 already 7
- **receptive field** = the part of the input that one value can "see"
- pooling and stride make it grow much faster
- deep layers see large parts of the image → they can recognise whole objects

Cheat sheet: CNN layers at a glance

layer	what it does	shape	learns weights?
`nn.Conv2d(c_in, c_out, k, padding=k//2)`	slides `c_out` small filters over the image	channels → `c_out`, size stays	yes
`nn.ReLU()`	sets negative values to 0	unchanged	no
`nn.MaxPool2d(2)`	keeps the maximum of every 2 × 2 window	size halves (using size 2)	no
`nn.BatchNorm2d(c)`	brings every channel to the same scale (day 3)	unchanged	yes (a few)
`nn.Dropout(p)`	switches off random values in training	unchanged	no
`nn.Flatten()`	turns the image into one long row	(c, h, w) → $c * h * w$	no
`nn.Linear(n, classes)`	the classifier at the end	→ number of classes	yes

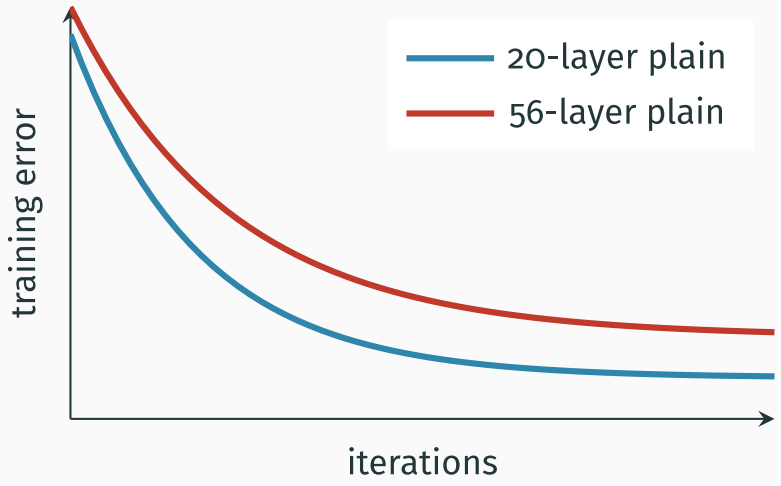
🔑 Key takeaway

A CNN = (conv → ReLU → pool) several times, then flatten → linear. Print the shape after every layer.

Deeper networks and object detection

Why deeper is not always better - and how to find objects

Deeper is not always better



schematic, after He et al. (2016)

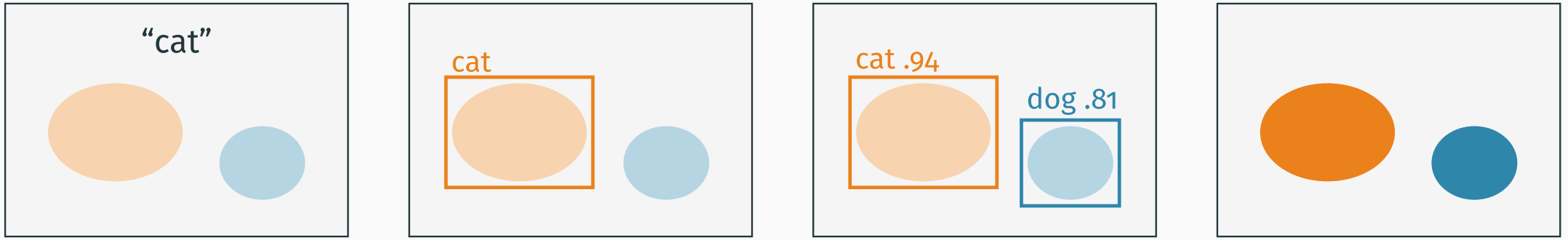
- a 56-layer network has a **higher training error** than a 20-layer network
- strange: the deep network could simply copy the small one and do "nothing" in the extra layers
- but for many layers, learning "do nothing" is hard
- the fix: a **shortcut** that lets the input skip the layers - ResNet, the standard today (day 5)

❓ Think about it

Is this overfitting? How can you tell from the picture?

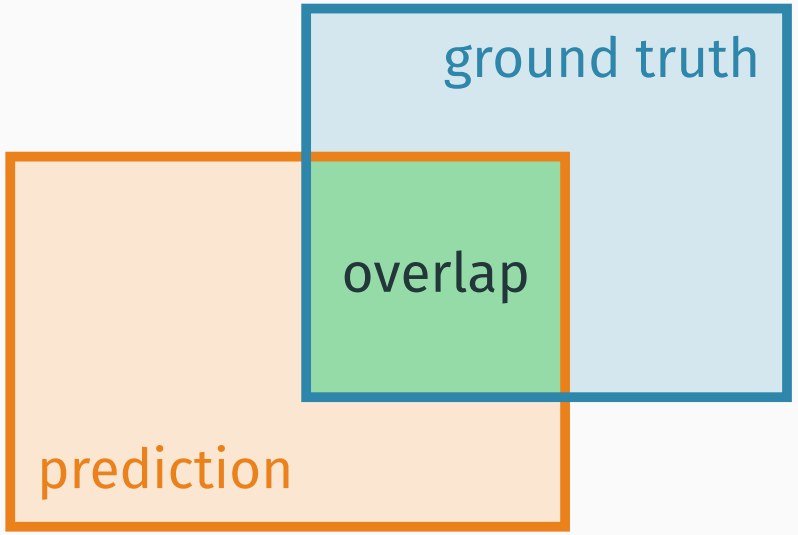
He et al., CVPR 2016

Beyond classification: object detection



- **classification**: one label for the whole image ("cat")
- **detection**: find every object - a **box** + a class + a score per object
- **segmentation**: a class for every pixel
- used in self-driving cars and medical images (e.g. finding lesions)

How good is a box? Intersection over Union (IoU)

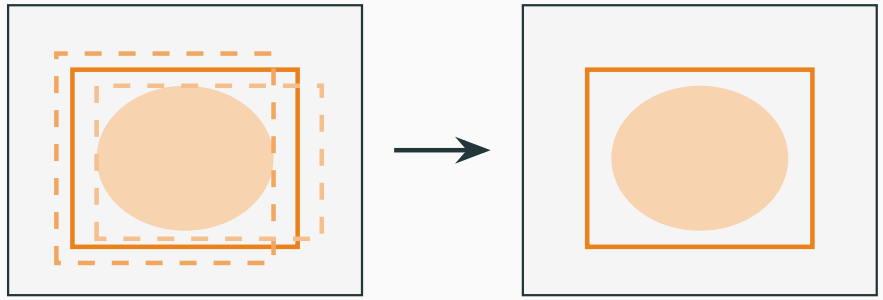


- **IoU** = overlap ÷ total area covered by both boxes
- 1 = perfect, 0 = no overlap
- a box counts as **correct** if IoU ≥ 0.5 (a common rule)

❓ Think about it

Two boxes of the same size, one shifted by half its width. What is the IoU? (Hint: it is not 0.5.)

Many boxes for one object: non-maximum suppression



- a detector often finds the same object several times
- **NMS**: keep the box with the highest score, delete all boxes that overlap it strongly (IoU > 0.5), repeat
- **YOLO** ("you only look once"): one CNN predicts all boxes in one go → real time, e.g. 45 images per second

```
from torchvision.ops import nms
keep = nms(boxes, scores, iou_threshold=0.5)
```

Redmon et al., CVPR 2016 (YOLO)

Before you train: three sanity checks

1. **shapes**: send a batch of 2 through the model, print the shape after every layer
2. **labels**: how many examples per class are in each split? (imbalance - day 3)
3. **overfit one batch**: train on one batch only - the loss must go to almost 0. If not, there is a bug.

```
x, y = next(iter(train_loader))      # ONE batch
opt = torch.optim.Adam(model.parameters())
for step in range(200):
    loss = loss_fn(model(x), y)
    opt.zero_grad()
    loss.backward()
    opt.step()
print(loss.item())                   # should be ~0
```

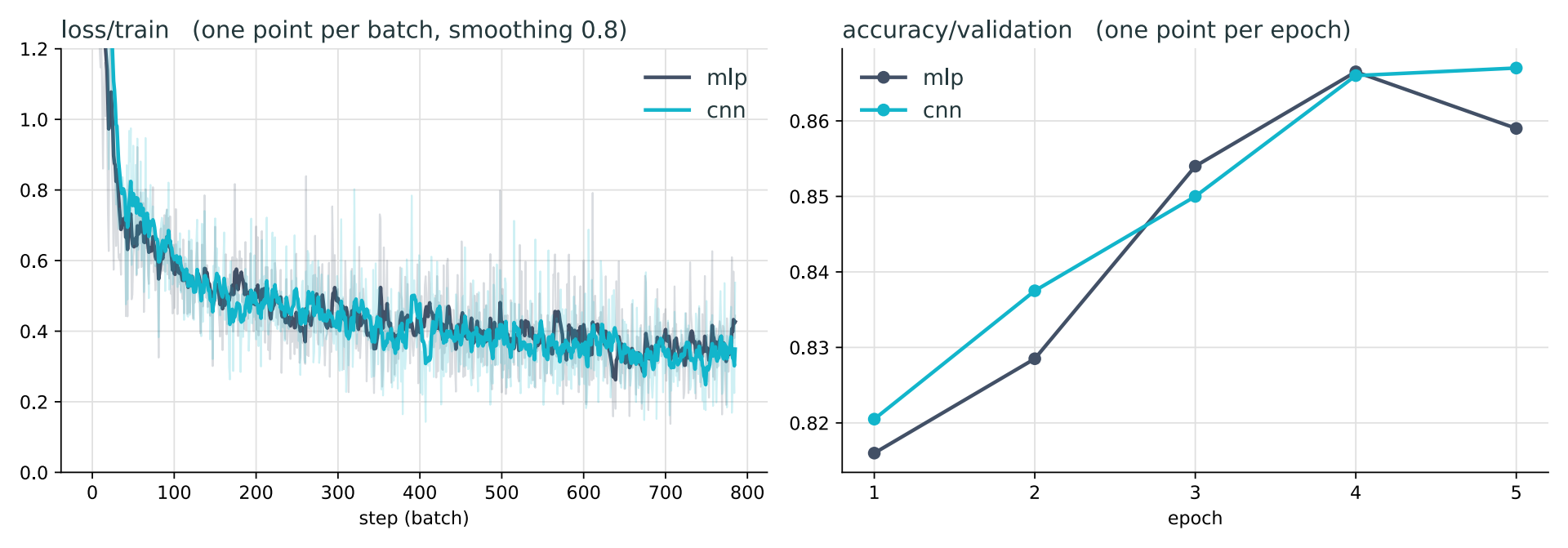
🔑 Key takeaway

Two minutes of checks save hours of searching for bugs - do them in the challenge, too.

Lab 4: build your own CNN

13:00-14:30 · task sheet at the end of the PDF

Watch your training: TensorBoard




```python
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter("runs/cnn")       # one folder per run
# inside the training loop:
writer.add_scalar("loss/train", loss.item(), step)
writer.add_scalar("accuracy/validation", acc, epoch)
```

```
%load_ext tensorboard                    # in Colab / Jupyter
%tensorboard --logdir runs
```

- built into PyTorch and Colab - every run is one coloured line
- in the lab: task 1.5 (MLP vs CNN) · for the challenge: W&B, MLflow or ClearML

Lab 4: write it yourself - CNNs for grey and colour images

💻 `day4_cnn.ipynb` in Colab · CPU is enough · first run: about 2 min download

Part 1 - a CNN for FashionMNIST (~35 min)

- 1.1 shapes by hand: what comes out of every layer?
- 1.2 build the CNN from the plan (`nn.Sequential`)
- 1.3 train it - better than the MLP from day 3?
- 1.4 look inside: learned filters and feature maps
- 1.5 watch the training in TensorBoard

Part 2 - colour images: CIFAR-10 (~30 min / homework)

- 2.1 shapes by hand with 3 input channels
- 2.2 build the CIFAR CNN, 2.3 train it
- 2.4 data augmentation against overfitting - compare in TensorBoard

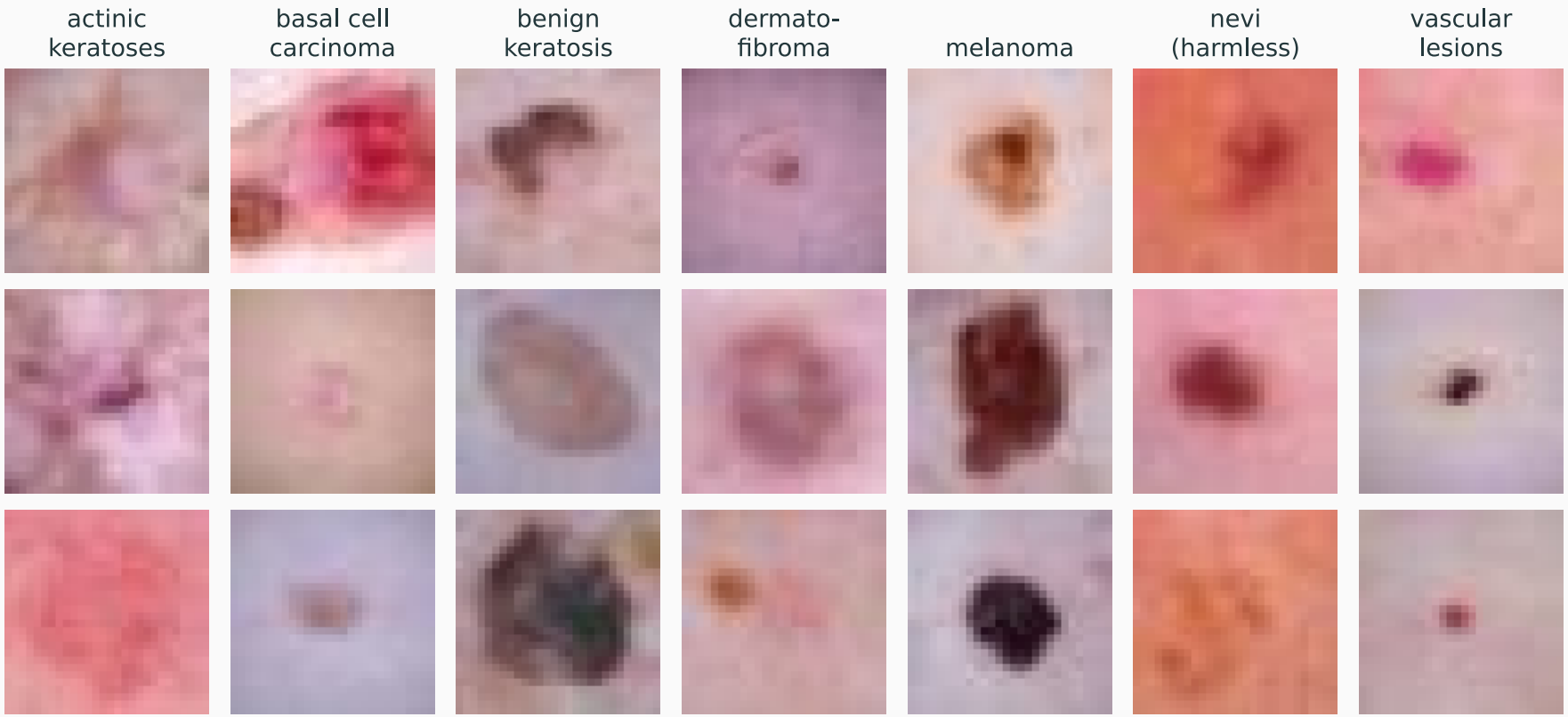
👥 How it works

In pairs. Plan → write the code → 💡 hint → ✅/❌ check. What you do not finish is homework.

The challenge

Your exam: a Model for skin lesions - built, tested, deployed and presented in teams of two

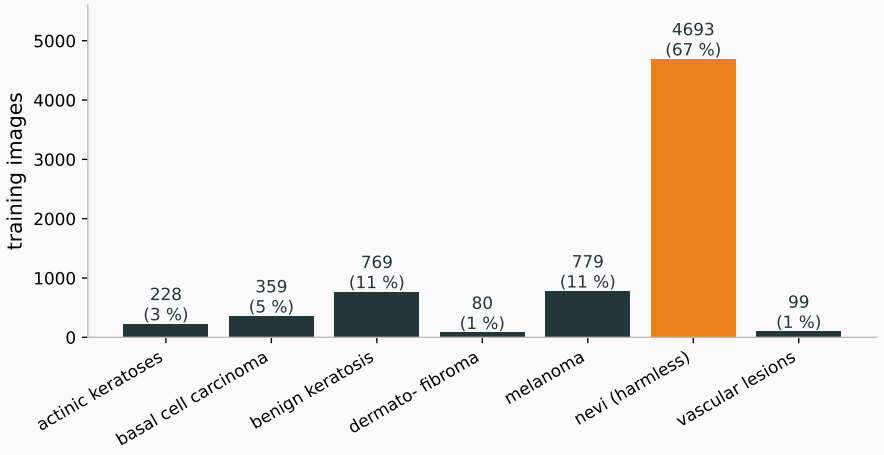
The task: recognise 7 kinds of skin lesions



- **DermaMNIST** (MedMNIST v2, based on the HAM10000 dermatoscopy images): 28 × 28 colour images, 7 classes
- official splits: **7 007 training**, **1 003 validation**, 2 005 test images - `pip install medmnist`
- your job: train a Model, optimize the model based on recent literature, and show what you tried - **in teams of two**

Yang et al., Scientific Data 2023 (MedMNIST v2); licence CC BY-NC 4.0

The main metric: balanced accuracy



- **67 %** of the images are harmless moles - a model that always says "nevus" has 67 % accuracy and finds **no melanoma**
- **balanced accuracy** = the mean sensitivity over the 7 classes - every class counts the same (day 3)
- also report macro-F1 and the sensitivity per class
- the baseline: accuracy **0.75**, but balanced accuracy only **0.44** - it finds almost no dermatofibroma

What you do: A → B

A - the baseline

- run and understand the starter notebook: data, baseline CNN, training loop, evaluation, export
- describe the data (class counts, examples) and the baseline results

B - improve it

- at least **two** well-motivated improvements by you: class weights, augmentation, a bigger CNN (BatchNorm, dropout, more blocks), a learning-rate schedule, ...
- at least **two** motivated by a paper or another good source - cite it! - explain why you think it helps!

Compare fairly (day 3)

- change **one thing at a time**
- every variant with **≥ 3 seeds**: mean ± std of the balanced accuracy on the **validation** images
- the **95 % bootstrap interval of the difference** to the baseline - the starter has `bootstrap_difference`
- **track every run** with Weights & Biases, MLflow or ClearML (free) - a screenshot goes into the report / presentation

🔑 Key takeaway

A clean comparison of a modest model is worth more than a lucky top score.

What you hand in - and the rules

Hand in

- `model.pt` - exported with TorchScript: input `(N, 3, 28, 28)`, normalised like in the starter → output logits `(N, 7)`
- **code** (notebook or Git repo) that reproduces your model
- **web app** from our Gradio template: upload an image → top-3 classes - shown **live**
- **short report, 2-4 pages, IEEE format** (LaTeX template provided): data, baseline, experiments, results table, discussion, AI use statement, **who did what** (at the end)
- **poster** (printed, portrait, A3, A2 or A1): problem, model, results table, one key figure
- **presentation of the poster**, about 12 min + questions, on 11.11.2026

Rules

- **train** only on train, **choose** on val
- the official **test** split: only to report your final results - never train on it
- keep the input format of the starter
- any compute is fine (Colab, own GPU, cluster) - but **your code must run**: a fresh run reproduces your model
- AI tools are allowed - **if you say how** you used them (report). Undisclosed use = attempt to deceive
- both team members can explain every part of the code - the report ends with a short statement of who did what

Timeline and grading

when	what
today	kick-off: teams, starter notebook, first baseline run
4 + 5 Nov	project days: work on your challenge in class (with help)
10 Nov	send me your `model.pt` · print your poster
11 Nov	hand in code, app, report · poster presentations + leaderboard reveal

part	weight
A: baseline and data	10 %
B: improvements and reasoning	25 %
evaluation: seeds, interval, metrics	20 %
report	15 %
presentation, poster and questions	15 %
app	5 %
code quality	10 %

The leaderboard rank gives at most a **small bonus**. All details: `handout.pdf` in the course repository.

Let's look at the starter code together

💻 `day4_challenge_starter.ipynb` - open it in Colab, run it along with me

section	what it does	what to look at
setup	installs `medmnist`, loads the data, normalises with `MEAN` / `STD`	the shape `(N, 3, 28, 28)`
1. look at the data	class counts, example images	the imbalance
2. the baseline CNN ("A")	`nn.Sequential` + the training loop from day 3	the knobs you can turn
3. evaluate	balanced accuracy, macro-F1, confusion matrix, sensitivity per class	which classes are missed?
4. export	`torch.jit.script` → `model.pt`, reload, same predictions	this file is what you hand in
5. your turn ("B")	class weights, augmentation, 3 seeds, `bootstrap_difference`, tracking tools (W&B, MLflow, ClearML)	your first improvement

Your turn: teams and the first run

1. form **teams of two** - write your names on the list
2. open the starter notebook in Colab and run it from top to bottom
3. write down your **validation balanced accuracy** of the baseline
4. agree on **one first idea** for B - and which source supports it

👥 In pairs (15 min)

Questions now - or in the Q&A at the end. The handout has all details.

Wrap-up

Summary

- **Convolution**: a small, shared filter slides over the image or signal - few weights, knows neighbourhoods
- **Shapes**: padding keeps the size, stride and pooling make it smaller - print the shape after every layer
- **A CNN**: (conv → ReLU → pool) several times, then flatten → linear
- **Deeper is not always better**: very deep plain networks train badly - shortcuts (ResNet) fix it (day 5)
- **Detection**: boxes, IoU, NMS
- **Before training**: print the shapes, check the labels, overfit one batch

Lab 4 – Write it yourself: CNNs

Notebook: `day4_cnn.ipynb` (course repository) · open it in Colab · CPU is enough · work in **pairs** · what you do not finish in class is **homework**.

How every writing task works

1. read the **plan** (a small table on this sheet and in the notebook),
2. write your code below the numbered steps – where it says `pass # YOUR CODE HERE`,
3. stuck? open **Hint** in the notebook,
4. run the **check cell**: ✔ = correct, ✘ = it tells you what is wrong,
5. tick the box ☐ of the task on this sheet.

First run the two **setup cells**. Something looks strange? *Runtime* → *Restart session*, then the setup cells again.

Part 1 – A CNN for clothes images

FashionMNIST · about 35 min

☐ 1.1 Shapes by hand

10 min

Before you build the CNN: write down the output shape after every layer for a batch of 8 images (`8, 1, 28, 28`). Then type them into the list `my_shapes` in the notebook and run the check.
Rules: `padding=1` with a 3×3 kernel keeps the size · `MaxPool2d(2)` halves height and width · a conv layer sets the channels to its *out* number · `Flatten` multiplies everything except the batch.

#	layer	in	out	kernel	note	output shape
1	`nn.Conv2d`	1	16	3	`padding=1`	`(8, 16, 28, 28)`
2	`nn.ReLU`					
3	`nn.MaxPool2d`			2		
4	`nn.Conv2d`	16	32	3	`padding=1`	
5	`nn.ReLU`					
6	`nn.MaxPool2d`			2		
7	`nn.Flatten`					
8	`nn.Linear`	1568	10		one score per class	

☐ 1.2 The CNN from the plan

10 min

Build the network of 1.1 with `nn.Sequential(...)`, e.g. `nn.Conv2d(1, 16, kernel_size=3, padding=1)`, `nn.MaxPool2d(2)`, `nn.Flatten()`.
Check: layers and sizes, parameter count, output shape `(8, 10)`.
Question: the MLP of Lab 3 had 235 146 parameters. How many does the CNN have – and in which layer are most of them?

☐ 1.3 Train it — 8 min

Train the CNN with the given `train_model` (your function from Lab 3) and compare with the MLP of Lab 3 (validation accuracy 0.836 after 3 epochs).
Question: why is the CNN better – with more than 10× fewer parameters?

☐ 1.4 Look inside — 7 min

A helper shows the 16 learned 3×3 filters of the first layer and what each makes of one image. Try other images (`IMAGE_NR = 1, 2, 3, ...`).
Question: which filters look like the hand-made *edge kernels* from the slides? What do their feature maps show?

☐ 1.5 Watch the training in TensorBoard — 10 min

Run the cell that trains the MLP of Lab 3 and your CNN again (5 epochs each, about 1 minute) and writes every step to TensorBoard. Then run the TensorBoard cell: tab **SCALARS** (or **TIME SERIES**), smoothing about 0.8.
Question: which model learns faster, and which one is better on the validation images?

Part 2 – Colour images: a CNN for CIFAR-10 — CIFAR-10 · about 30 min

32×32 colour photos of 10 kinds of objects (planes, cars, birds, cats, ...) – every image has **3 channels** (red, green, blue): shape `(3, 32, 32)`. 10 000 training and 2 000 validation images; the download (170 MB) takes about 1 minute.

☐ 2.1 Shapes by hand — 8 min

Output shape after every layer for a batch of 8 images `(8, 3, 32, 32)`. Same rules as in 1.1.

#	layer	in	out	kernel	note	output shape
1	`nn.Conv2d`	3	16	3	`padding=1`	`(8, 16, 32, 32)`
2	`nn.ReLU`					
3	`nn.MaxPool2d`			2		
4	`nn.Conv2d`	16	32	3	`padding=1`	
5	`nn.ReLU`					
6	`nn.MaxPool2d`			2		
7	`nn.Conv2d`	32	64	3	`padding=1`	
8	`nn.ReLU`					
9	`nn.MaxPool2d`			2		
10	`nn.Flatten`					
11	`nn.Linear`	1024	10		one score per class	

☐ **2.2** The CIFAR CNN from the plan — 7 min

Write `make_cifar_cnn()`: the 11 layers of 2.1 in `nn.Sequential(...)` – the first layer gets **3** input channels: `nn.Conv2d(3, 16, kernel_size=3, padding=1)`.
Check: layers and sizes, parameter count, output shape `(8, 10)`.
Question: why does the first conv layer have more weights than in 1.2?

☐ **2.3** Train it — 7 min

The given `train_model` trains the CNN on CIFAR-10 for 15 epochs, about 1 minute (TensorBoard run `cifar`); the cell also prints the accuracy on 2 000 **training** images.
Question: why is CIFAR-10 much harder than FashionMNIST – and what does the gap between training and validation accuracy tell you?

☐ **2.4** Data augmentation against overfitting — 8 min

Train again with `augment=True` (random crop + left-right flip, TensorBoard run `cifar_aug`) and compare `cifar` and `cifar_aug` in TensorBoard (`accuracy/train` and `accuracy/validation`).
Question: what happens to the gap between training and validation accuracy?

☐ ★ **Bonus** Overfit one batch — homework

The most useful sanity check before a long training run: train 200 steps on **one** batch of 64 images. Write the loop (forward, loss, `zero_grad`, `backward`, `step`, append the loss). The loss must go to almost 0 – otherwise there is a bug in the pipeline.